模式识别与机器学习 (6)

贝叶斯学习方法

左旺孟

哈尔滨工业大学计算机学院
机器学习研究中心
综合楼712
cswmzuo@gmail.com
13134506692

学习内容：贝叶斯学习基础

- 1. 参数估计
  - 最大似然估计 (ML)
  - 最大化后验估计 (贝叶斯估计)
  - 完全贝叶斯估计
- 2. 非参数估计
- 3. 朴素贝叶斯分类器
- 4. 隐马尔可夫模型
- …

应用

分类器讨论

$$P(\omega_i|\mathbf{x})=\frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{\sum_{j=1}^{2}P(\mathbf{x}|\omega_j)P(\omega_j)}$$

3. 贝叶斯学习：根据训练样本对类别先验概率和类条件概率进行建模，并进一步构建分类器

$$\begin{cases}\text{如果}P(\omega_1|\mathbf{x})>P(\omega_2|\mathbf{x}), & \mathbf{x}\in\omega_1\\ \text{如果}P(\omega_1|\mathbf{x})<P(\omega_2|\mathbf{x}), & \mathbf{x}\in\omega_2\end{cases}$$

$$F(\mathbf{x})=\log\left(\frac{P(\omega_1|\mathbf{x})}{P(\omega_2|\mathbf{x})}\right)\begin{cases}\geq 0, & \mathbf{x}\in\omega_1\\ <0, & \mathbf{x}\in\omega_2\end{cases}$$

2. 判别学习：根据训练样本对类别后验概率进行建模

判别函数：$g_i(\mathbf{x})=P(\omega_i|\mathbf{x})$

1. 判别函数：根据训练样本学习直接学习函数$F(\mathbf{x})$

判别函数：$g(\mathbf{x})=F(\mathbf{x})$

语言大模型：ChatGPT

- Word i: $\boldsymbol{x}_i$
- LLM:

$$\underbrace{p_\theta(x) = p_\theta(x^1) \prod_{i=2}^{L} p_\theta(x^i \mid x^1, \ldots, x^{i-1})}_{\text{Autoregressive formulation}},$$

- Diffusion Language Model

Prompt: "Explain what artificial intelligence is."

3. NaiveBayes

- 联合概率密度估计
- NaiveBayes
  - 离散
  - 连续：高斯NaiveBayes (GNB)
- 应用举例

联合概率密度估计

- 贝叶斯定理 $\mathbf{x}=[x_1,\ldots,x_d]^t$

$$P(\omega_i|\mathbf{x})=\frac{P(\mathbf{x}|\omega_i)P(\omega_i)}{\sum_{j=1}^{2}P(\mathbf{x}|\omega_j)P(\omega_j)}$$

- 联合概率密度
  - 如果样本数较少，特征维数较高，概率密度的估计会非常困难（过拟合）

联合概率密度估计

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

- 样本：X = <sky =sun, Temp =cold, humid =high, wind=strong>
- 试估计P(X|Y)
- 如果能准确估计P(X|Y)，则可对测试样本按照贝叶斯公式进行分类

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

条件独立性

- 如果样本数较少，特征维数较高，概率密度的估计会非常困难。
- 独立性 $P(AB) = P(A)P(B) \Rightarrow P(A|B) = P(A)$
- NaiveBayes基本假设
  - 条件独立

$$P(X_1 \dots X_n|Y) = \prod_i P(X_i|Y)$$

条件独立性

- 定义：给定Z值的条件下，如果X的概率分布与Y独立，即

$$(\forall i,j,k)P(X=x_i \mid Y=y_j, Z=z_k)=P(X=x_i \mid Z=z_k)$$

则称给定Z的条件下X和Y条件独立。

上式可以简写为

$$P(X \mid Y,Z)=P(X \mid Z)$$

进一步，有

$$P(X,Y \mid Z)=P(X \mid Z)P(Y \mid Z)$$

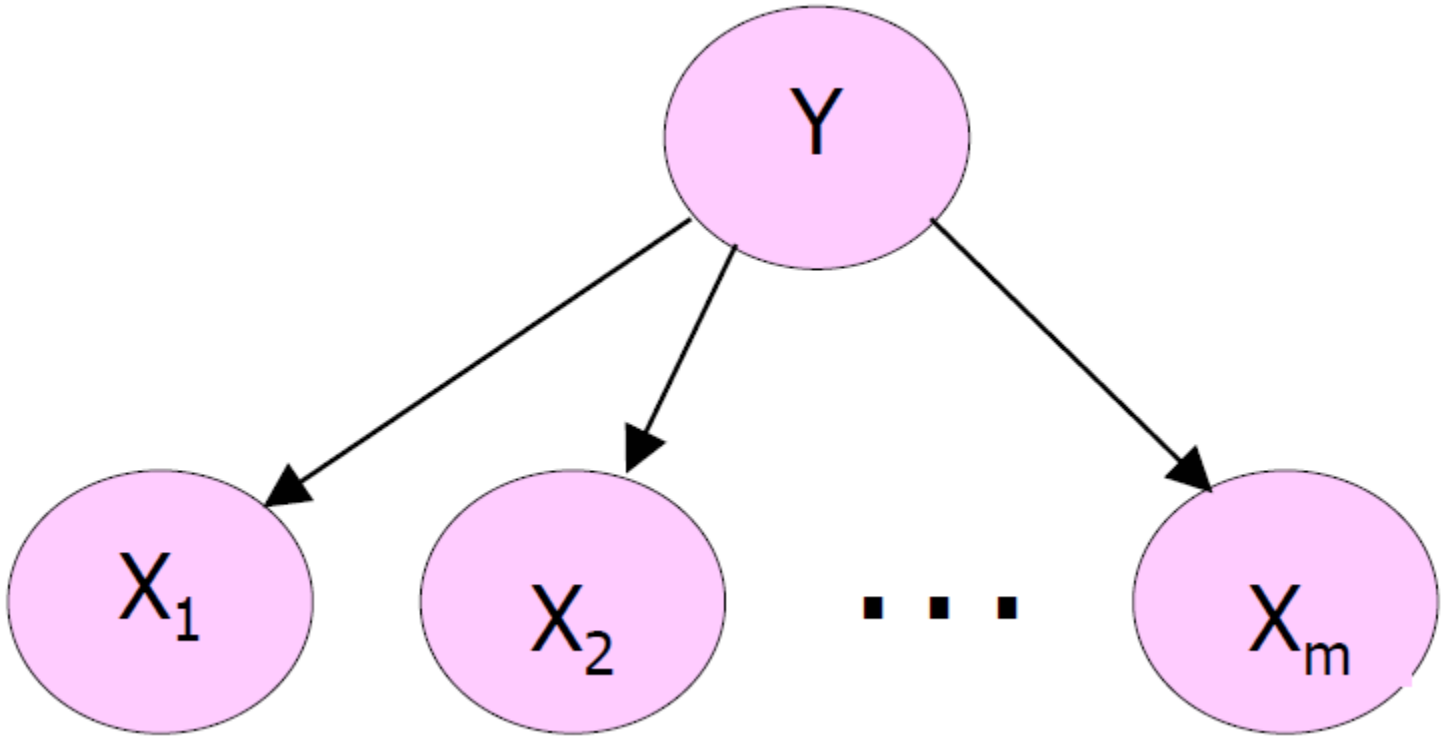
NaiveBayes

- 贝叶斯方法
  - 需要估计类条件概率密度, 但直接估计非常困难
- NaiveBayes
  - 假设给定Y的条件下不同特征之间条件独立

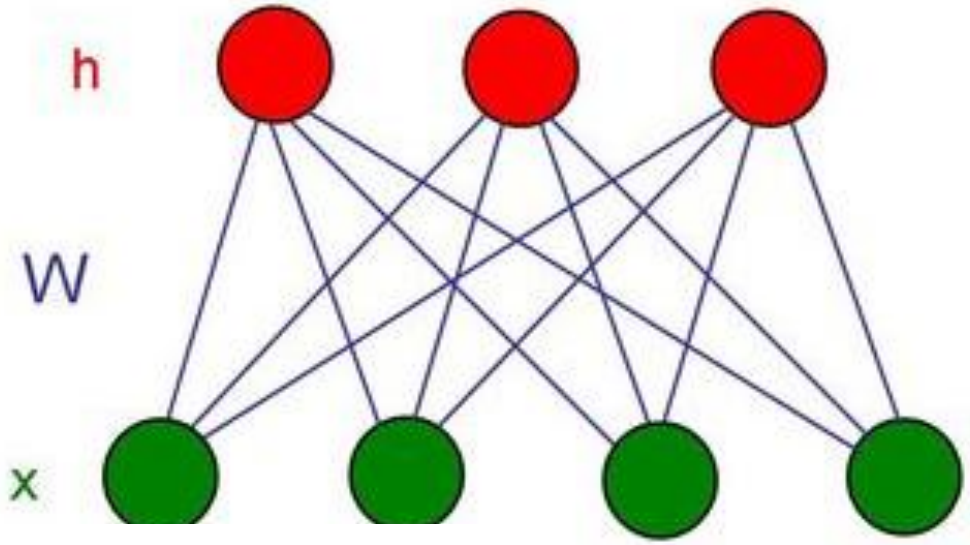
$$P(X_1 \ldots X_n|Y) = \prod_i P(X_i|Y)$$

  - 基于上述假设可以大大减少待估计参数的数目
  - 合理性和简单性的折中

图示



Restricted Boltzmann Machine (RBM)



$$P(v|h) = \prod_{i=1}^{m} P(v_i|h). \qquad P(h|v) = \prod_{j=1}^{n} P(h_j|v).$$

$$P(h_j = 1|v) = \sigma\left(b_j + \sum_{i=1}^{m} w_{i,j} v_i\right) \text{ and } P(v_i = 1|h) = \sigma\left(a_i + \sum_{j=1}^{n} w_{i,j} h_j\right)$$

Hinton, G. E.; Salakhutdinov, R. R. (2006). "Reducing the Dimensionality of Data with Neural Networks" (PDF). Science. 313 (5786): 504–507.

NaiveBayes分类

- 贝叶斯方法

$$P(Y = y_k|X_1 \dots X_n) = \frac{P(Y = y_k)P(X_1 \dots X_n|Y = y_k)}{\sum_j P(Y = y_j)P(X_1 \dots X_n|Y = y_j)}$$

- NaiveBayes

$$P(Y = y_k|X_1 \dots X_n) = \frac{P(Y = y_k)\prod_i P(X_i|Y = y_k)}{\sum_j P(Y = y_j)\prod_i P(X_i|Y = y_j)}$$

  - 分类规则

$$X^{new} = \langle X_1, \dots, X_n \rangle$$

$$Y^{new} \leftarrow \arg\max_{y_k} P(Y = y_k)\prod_i P(X_i|Y = y_k)$$

NaiveBayes算法

- 训练

Train Naïve Bayes (examples)

for each* value y_k

estimate $\pi_k \equiv P(Y = y_k)$

for each* value x_{ij} of each attribute X_i

estimate $\theta_{ijk} \equiv P(X_i = x_{ij}|Y = y_k)$

- 分类

Classify (X^{new})

$$Y^{new} \leftarrow \arg\max_{y_k} \; P(Y = y_k) \prod_i P(X_i|Y = y_k)$$

* 需要满足非负和归一化条件

特征为离散情况下的参数估计

- 概率分布：n值分布
- 最大似然估计 (ML)

$$\hat{\pi}_k = \hat{P}(Y = y_k) = \frac{\#D\{Y = y_k\}}{|D|}$$

$$\hat{\theta}_{ijk} = \hat{P}(X_i = x_{ij}|Y = y_k) = \frac{\#D\{X_i = x_{ij} \wedge Y = y_k\}}{\#D\{Y = y_k\}}$$

- 最大后验概率估计 (MAP)

$$\hat{\pi}_k = \hat{P}(Y = y_k) = \frac{\#D\{Y = y_k\} + l}{|D| + lR}$$

$$\hat{\theta}_{ijk} = \hat{P}(X_i = x_{ij}|Y = y_k) = \frac{\#D\{X_i = x_{ij} \wedge Y = y_k\} + l}{\#D\{Y = y_k\} + lM}$$

例：是否适合运动

- 训练数据

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

- 样本：X = <sky =sun, Temp =cold, humid =high, wind=strong>
  - 计算：

$$P(y)P(sun|y)P(cold|y)P(high|y)$$

$$P(n)P(sun|n)P(cold|n)P(high|n)$$

  - 分类

例：文本分类

- 将文本归为某一类
- 应用
  - 识别垃圾邮件
  - 网页分类

```
Path: cantaloupe.srv.cs.cmu.edu!das-news.harvard.e
From: xxx@yyy.zzz.edu (John Doe)
Subject: Re: This year's biggest and worst (opinic
Date: 5 Apr 93 09:53:39 GMT

I can only comment on the Kings, but the most
obvious candidate for pleasant surprise is Alex
Zhitnik. He came highly touted as a defensive
defenseman, but he's clearly much more than that.
Great skater and hard shot (though wish he were
more accurate). In fact, he pretty much allowed
the Kings to trade away that huge defensive
liability Paul Coffey. Kelly Hrudey is only the
biggest disappointment if you thought he was any
good to begin with. But, at best, he's only a
mediocre goaltender. A better choice would be
Tomas Sandstrom, though not through any fault of
his own, but because some thugs in Toronto decided
```

例：文本分类

aardvark	0
about	2
all	2
Africa	1
apple	0
anxious	0
...	
gas	1
...	
oil	1
...	
Zaire	0

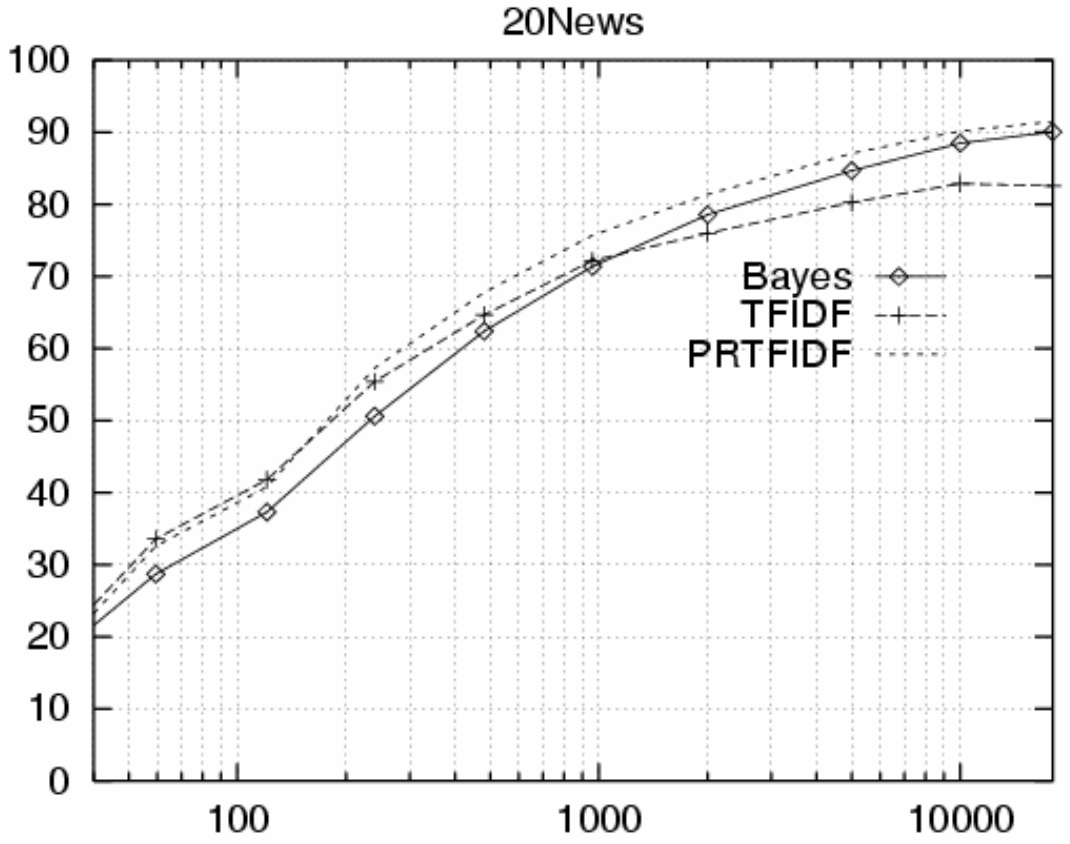
NaiveBayes

20 NewsGroups

- http://people.csail.mit.edu/jrennie/20Newsgroups/

comp.graphics	misc.forsale
comp.os.ms-windows.misc	rec.autos
comp.sys.ibm.pc.hardware	rec.motorcycles
comp.sys.mac.hardware	rec.sport.baseball
comp.windows.x	rec.sport.hockey
alt.atheism	sci.space
soc.religion.christian	sci.crypt
talk.religion.misc	sci.electronics
talk.politics.mideast	sci.med
talk.politics.misc	
talk.politics.guns	

Learning Curve for 20 Newsgroups



Accuracy vs. Training set size (1/3 withheld for test)

Note

- 停用词：统计词频时去掉the、and、of等常见词

- TF.IDF特征
  - TF：词i在文档中出现的频率
  - IDF（逆文档频率）：对词i在文档集中出现的频率求log。

连续特征

- 概率分布：正态分布
- 高斯NaiveBayes (GNB)

$$P(X_i = x \mid Y = y_k) = \frac{1}{\sigma_{ik}\sqrt{2\pi}} e^{\frac{-(x-\mu_{ik})^2}{2\sigma_{ik}^2}}$$

  - 为简化计算，可进一步假设
    - 方差与Y无关，即 σ_i
    - **方差与X无关，即 σ_k**
    - 方差与X和Y均无关，即 σ

连续特征：参数估计

- 最大似然估计

Maximum likelihood estimates:

$$\hat{\mu}_{ik} = \frac{1}{\sum_j \delta(Y^j = y_k)} \sum_j X_i^j \delta(Y^j = y_k)$$

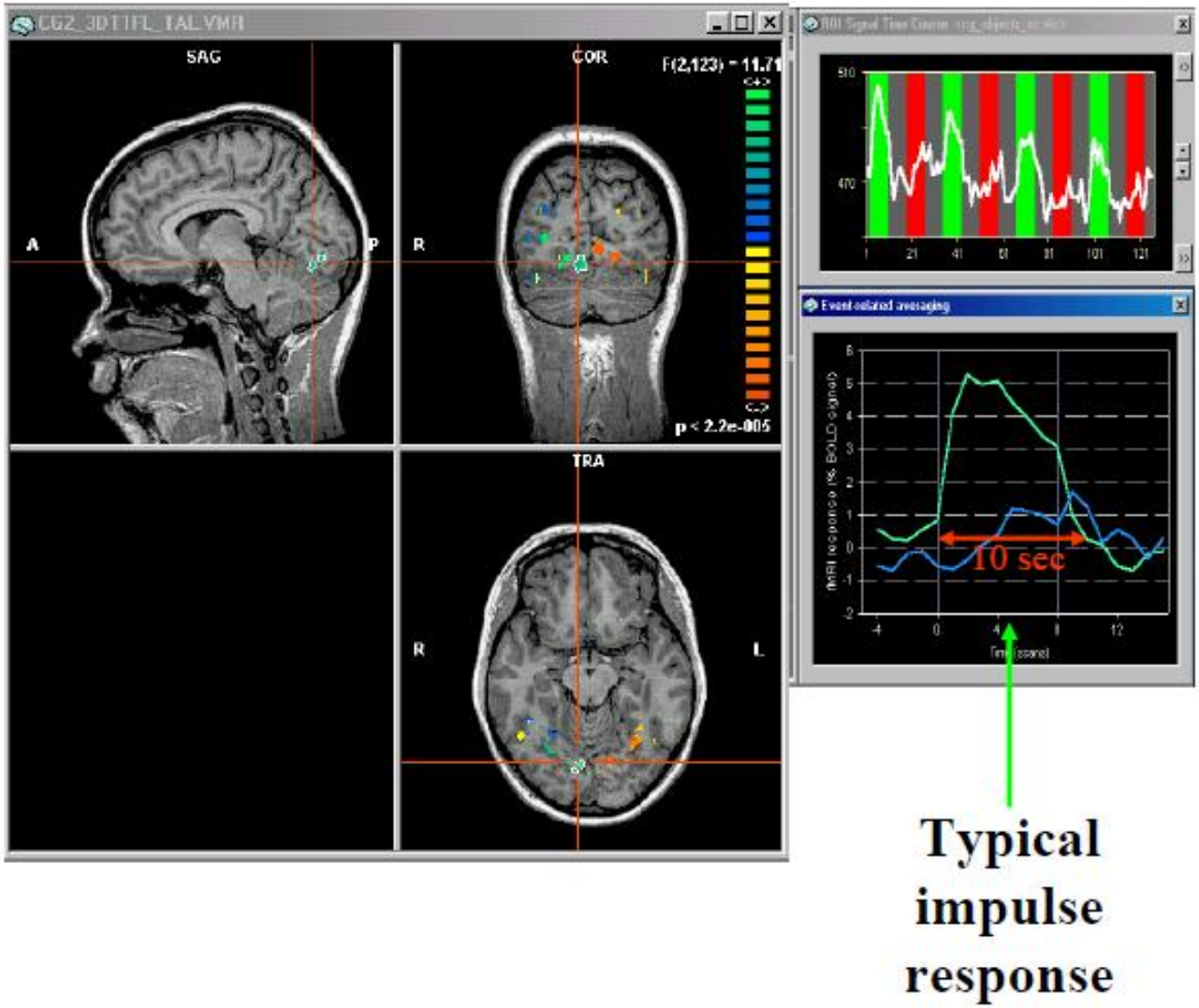
jth training example

$\delta(x)$=1 if x true, else 0

$$\hat{\sigma}_{ik}^2 = \frac{1}{\sum_j \delta(Y^j = y_k)} \sum_j (X_i^j - \hat{\mu}_{ik})^2 \delta(Y^j = y_k)$$

例：精神状态分类

- fMRI数据
  - 血氧水平相应数据
  - 反映脑区的活跃程度



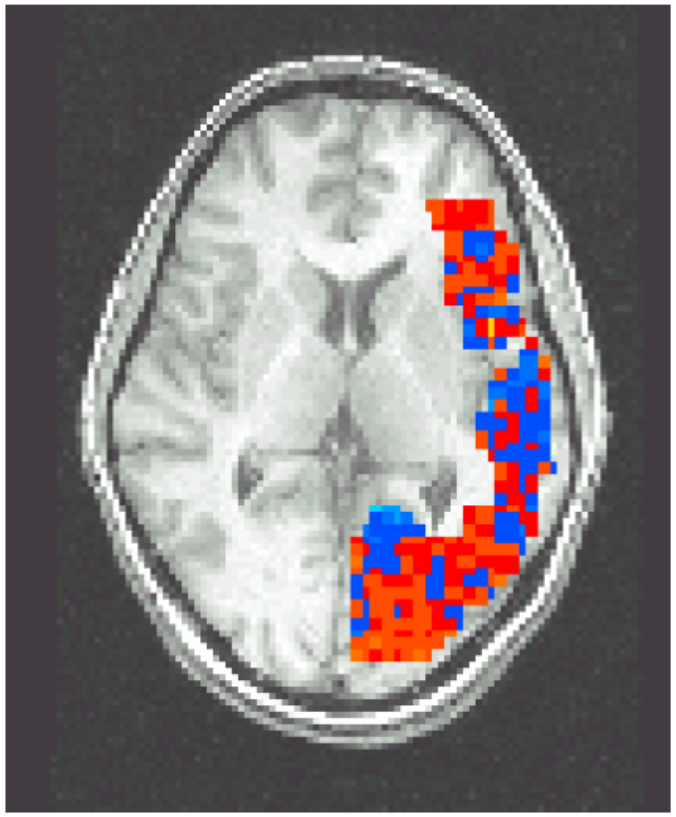
NaiveBayes模型

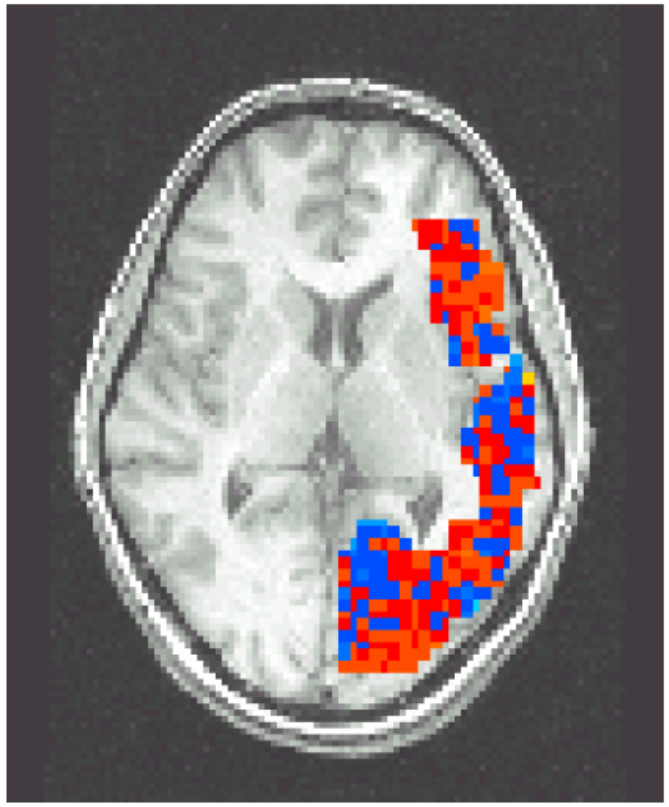
P(BrainActivity | WordCategory)

Pairwise classification accuracy: 85%

People words

Animal words



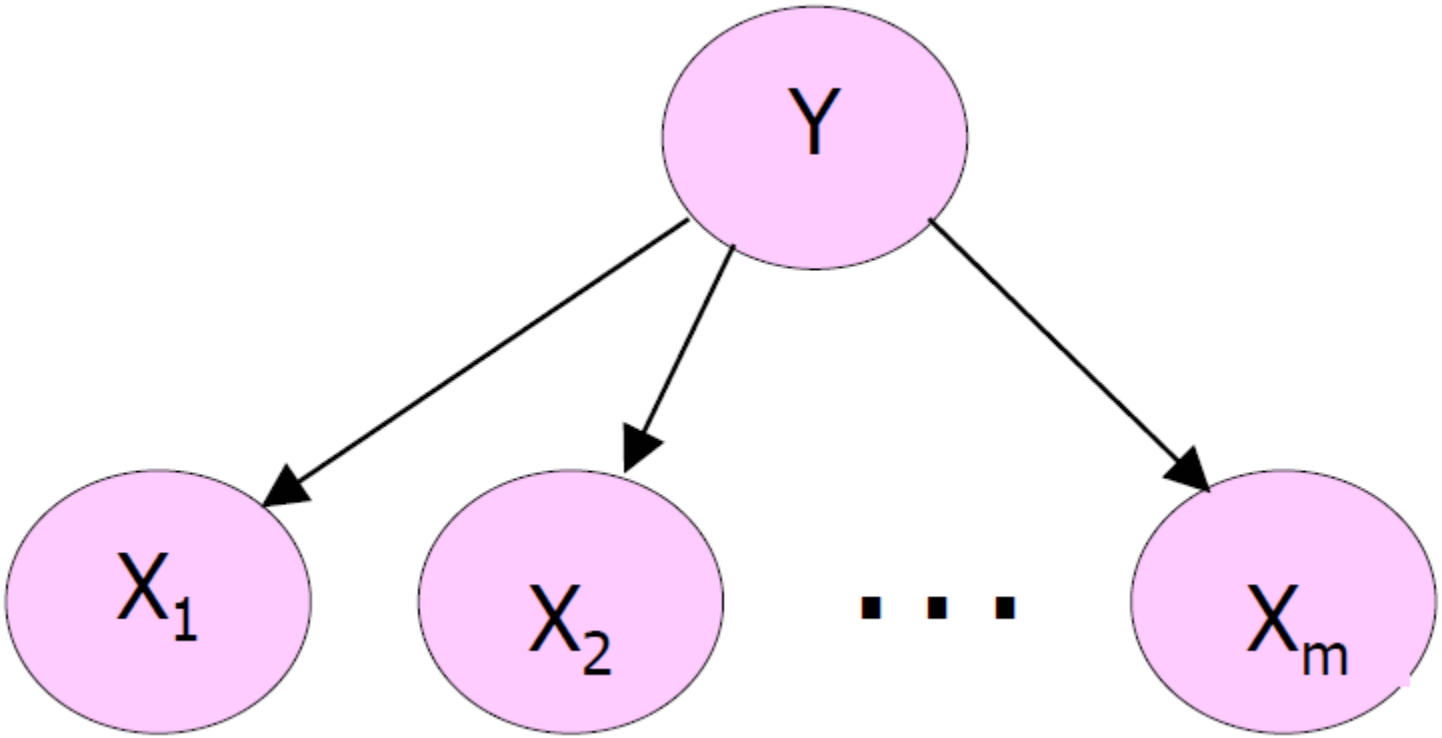


学习内容：贝叶斯学习基础

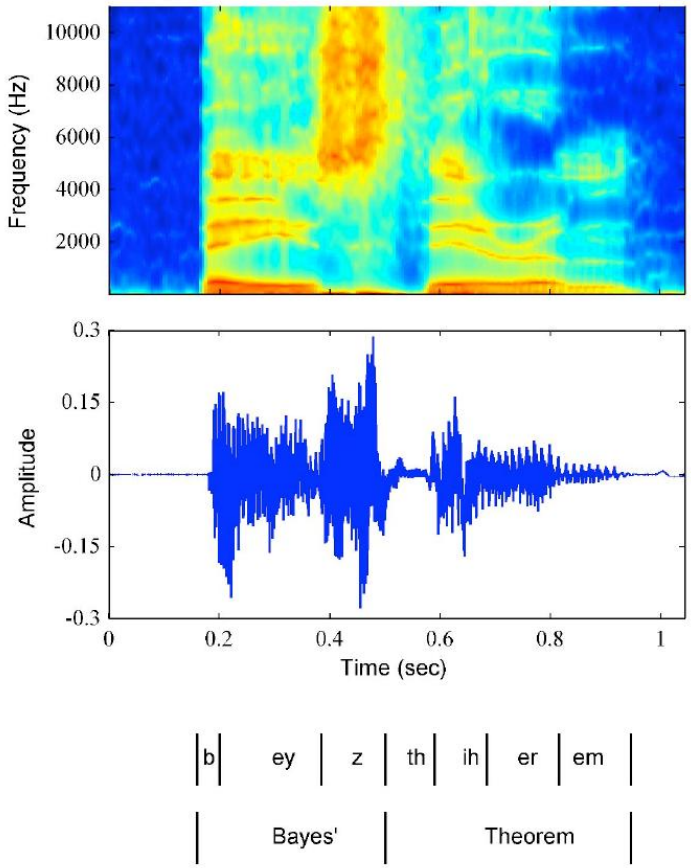
- 1. 参数估计
  - 最大似然估计 (ML)
  - 最大化后验估计 (贝叶斯估计)
  - 完全贝叶斯估计
- 2. 非参数估计
- 3. 朴素贝叶斯分类器
- 4. 隐马尔可夫模型
- …

应用

朴素贝叶斯分类器



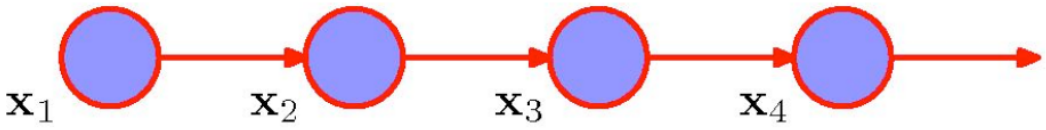
Example of a Spectrogram



- Example of a spectrogram of a spoken word 'Bayes theorem':
- Successive observations are highly correlated.

Markov Models

- The simplest model is the first-order Markov chain:



- The joint distribution for a sequence of N observations under this model is:

$$p(\mathbf{x}_1, ..., \mathbf{x}_N) = p(\mathbf{x}_1) \prod_{n=2}^{N} p(\mathbf{x}_n|\mathbf{x}_{n-1}).$$

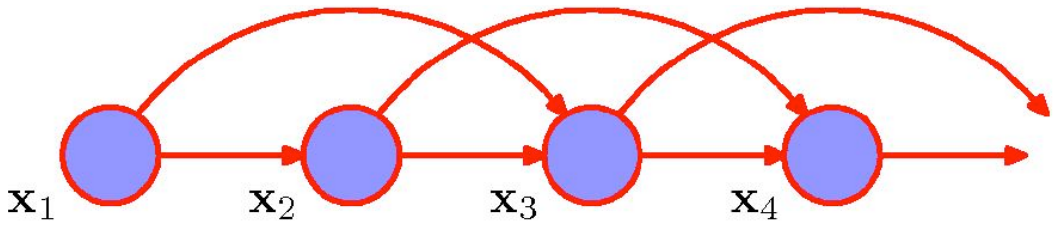
- From the d-separation property, the conditionals are given by:

$$p(\mathbf{x}_n|\mathbf{x}_1, ..., \mathbf{x}_{n-1}) = p(\mathbf{x}_n|\mathbf{x}_{n-1}).$$

- For many applications, these conditional distributions that define the model will be constrained to be equal.
- This corresponds to the assumption of a stationary time series.
- The model is known as homogenous Markov chain.

Second-Order Markov Models

- We can also consider a second-order Markov chain:



- The joint distribution for a sequence of N observations under this model is:

$$p(\mathbf{x}_1, ..., \mathbf{x}_N) = p(\mathbf{x}_1)p(\mathbf{x}_2|\mathbf{x}_1) \prod_{n=3}^{N} p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2}).$$

- We can similarly consider extensions to an M^{th} order Markov chain.
- Increased flexibility $\rightarrow$ Exponential growth in the number of parameters.
- Markov models need big orders to remember past "events".

Learning Markov Models

• The ML parameter estimates for a simple Markov model are easy.
Consider a k[th] order model:

$$p(\mathbf{x}_N, ..., \mathbf{x}_{k+1}|\mathbf{x}_1, .., \mathbf{x}_k) = \prod_{n=k+1}^{N} p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2}, .., \mathbf{x}_{n-k}).$$

• Each window of k + 1 outputs is a training case for the model.
$p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2}, .., \mathbf{x}_{n-k})$.

• Example: for discrete outputs (symbols) and a 2nd-order Markov model we can use the multinomial model:

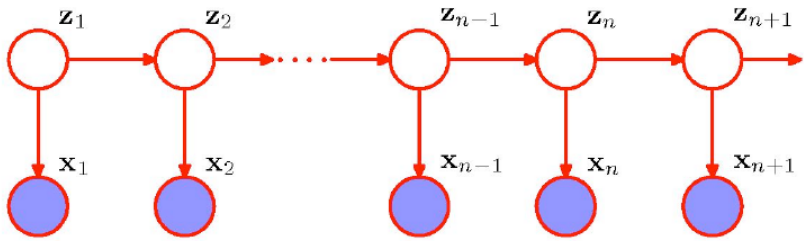
$$p(\mathbf{x}_n = m|\mathbf{x}_{n-1} = a, \mathbf{x}_{n-2} = b) = \alpha_{mab}.$$

• The maximum likelihood values for α are:

$$\alpha^*_{mab} = \frac{\text{num}[n,\ s.t.\ \mathbf{x}_n = m, \mathbf{x}_{n-1} = a, \mathbf{x}_{n-2} = b]}{\text{num}[n,\ s.t.\ \mathbf{x}_{n-1} = a, \mathbf{x}_{n-2} = b]}.$$

State Space Models

- How about the model that is not limited by the Markov assumption to any order.
- Solution: Introduce additional latent variables!



- Graphical structure known as the State Space Model.

- For each observation x_n, we have a latent variable z_n. Assume that latent variables form a Markov chain.
- If the latent variables are discrete → Hidden Markov Models (HMMs). Observed variables can be discrete or continuous.
- If the latent and observed variables are Gaussian → Linear Dynamical System.

Hidden Markov Model

- First order Markov chain generates hidden state sequence (known as transition probabilities):

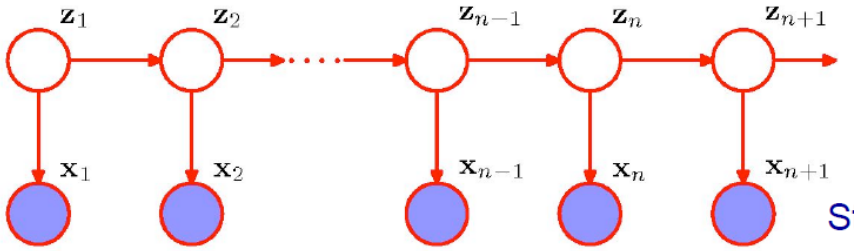
$$p(\mathbf{z}_n = k|\mathbf{z}_{n-1} = j) = A_{jk}, \qquad p(\mathbf{z}_1 = k) = \pi_k.$$

- A set of output probability distributions (**one per state**) converts state path into sequence of observable symbols/vectors (known as emission probabilities):

$$p(\mathbf{x}_n|\mathbf{z}_n, \boldsymbol{\phi}).$$

Gaussian, if **x** is continuous.

Conditional probability table if **x** is discrete.



State transition

Observation model

$$p(\mathbf{x}_1, ..., \mathbf{x}_N, \mathbf{z}_1, .., \mathbf{z}_N) = p(\mathbf{z}_1) \prod_{n=2}^{N} p(\mathbf{z}_n|\mathbf{z}_{n-1}) \prod_{n=1}^{N} p(\mathbf{x}_n|\mathbf{z}_n).$$

Transition Probabilities

- It will be convenient to use 1-of-K encoding for the latent variables.
- The matrix of transition probabilities takes form:

$$p(z_{nk} = 1 | z_{n-1,j} = 1) = A_{jk}, \quad \sum_k A_{jk} = 1.$$

- The conditionals can be written as:

$$p(\mathbf{z}_n | \mathbf{z}_{n-1}, A) = \prod_{k=1}^{K} \prod_{j=1}^{K} A_{jk}^{z_{n-1,j} z_{nk}}, \quad p(\mathbf{z}_1 | \pi) = \prod_{k=1}^{K} \pi_k^{z_{1k}}.$$

- We will focus on homogenous models: all of the conditional distributions over latent variables share the same parameters **A**.
- Standard mixture model for i.i.d. data: special case in which all parameters A_{jk} are the same for all j.
- Or the conditional distribution $p(z_n|z_{n-1})$ is independent of z_{n-1}.

Emission Probabilities

- The emission probabilities take form:

$$p(\mathbf{x}_n|\mathbf{z}_n,\boldsymbol{\phi}) = \prod_{k=1}^{K} p(\mathbf{x}_n|\boldsymbol{\phi}_k)^{z_{nk}}.$$

- For example, for a continuous **x**, we have

$$p(\mathbf{x}_n|\mathbf{z}_n,\boldsymbol{\phi}) = \prod_{k=1}^{K} \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)^{z_{nk}}.$$

- For the discrete, multinomial observed variable **x**, using 1-of-K encoding, the conditional distribution takes form:

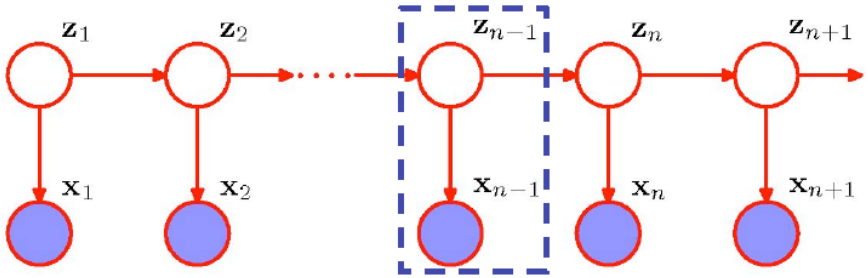
$$p(\mathbf{x}_n|\mathbf{z}_n,\boldsymbol{\phi}) = \prod_{i=1}^{D}\prod_{k=1}^{K} \mu_{ik}^{x_{ni}z_{nk}}.$$

HMM Model Equations

- The joint distribution over the observed and latent variables is given by:

$$p(\mathbf{X}, \mathbf{Z}|\theta) = p(\mathbf{z}_1|\boldsymbol{\pi}) \prod_{n=2}^{N} p(\mathbf{z}_n|\mathbf{z}_{n-1}, A) \prod_{n=1}^{N} p(\mathbf{x}_n|\mathbf{z}_n, \boldsymbol{\phi}),$$

where $\theta = \{\boldsymbol{\pi}, A, \boldsymbol{\phi}\}$ are the model parameters.



- Data are not i.i.d. Everything is coupled across time.

- Three problems: computing probabilities of observed sequences, inference of hidden state sequences, learning of parameters.

Maximum Likelihood for the HMM

- We observe a dataset **X** = $\{x_1,\ldots,x_N\}$.
- The goal is to determine model parameters $\theta = \{\boldsymbol{\pi}, A, \boldsymbol{\phi}\}$.
- The probability of observed sequence takes form:

$$p(\mathbf{X}|\theta) = \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z}|\theta).$$

$$p(\text{observed sequence}) = \sum_{\text{all paths}} p(\text{observed outputs, state paths}).$$

- In contrast to mixture models, the joint distribution p(**X**,**Z** | θ) does not factorize over n.
- It looks hard: N variables, each of which has K states. Hence N^K total paths.
- Remember inference problem on a simple chain.

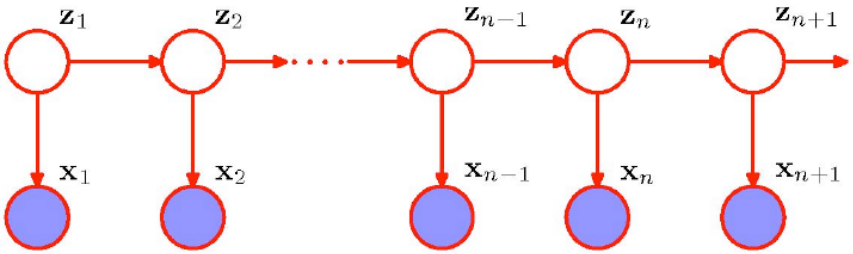
Probability of an Observed Sequence

- The joint distribution factorizes:

$$p(\mathbf{X}|\theta) = \sum_{\mathbf{Z}} p(\mathbf{X},\mathbf{Z}|\theta) = \sum_{\mathbf{z}_1,\ldots,\mathbf{z}_n} p(\mathbf{z}_1,\mathbf{x}_1) \prod_{n=2}^{N} p(\mathbf{z}_n|\mathbf{z}_{n-1}) p(\mathbf{x}_n|\mathbf{z}_n)$$

$$= \sum_{\mathbf{z}_1} p(\mathbf{z}_1) p(\mathbf{x}_1|\mathbf{z}_1) \sum_{\mathbf{z}_2} p(\mathbf{z}_2|\mathbf{z}_1) p(\mathbf{x}_2|\mathbf{z}_2) \ldots$$

$$\sum_{\mathbf{z}_N} p(\mathbf{z}_N|\mathbf{z}_{N-1}) p(\mathbf{x}_N|\mathbf{z}_N).$$



- Dynamic Programming: By moving the summations inside, we can save a lot of work.

EM algorithm

- We cannot perform direct maximization (no closed form solution):

$$p(\mathbf{X}|\theta) = \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z}|\theta).$$

- EM algorithm: we will derive efficient algorithm for maximizing the likelihood function in HMMs (and later for linear state-space models).

- E-step: Compute the posterior distribution over latent variables:

$$p(\mathbf{Z}|\mathbf{X}, \theta^{old}).$$

- M-step: Maximize the expected complete data log-likelihood:

$$Q(\theta, \theta^{old}) = \sum_{\mathbf{Z}} p(\mathbf{Z}|\mathbf{X}, \theta^{old}) \log p(\mathbf{X}, \mathbf{Z}|\theta).$$

- If we knew the true state path, then ML parameter estimation would be trivial.

- We will first look at the E-step: Computing the true posterior distribution over the state paths.

Inference of Hidden States

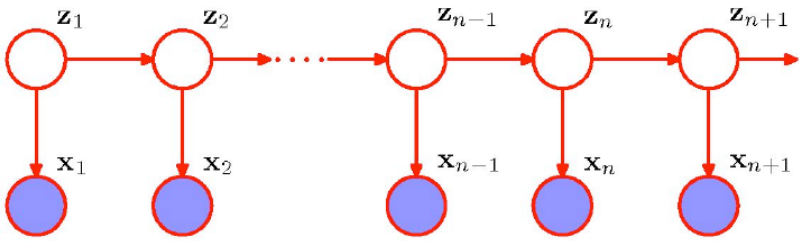
• We want to estimate the hidden states given observations. To start with, let us estimate a single hidden state:

$$\gamma(\mathbf{z}_n) = p(\mathbf{z}_n|\mathbf{X}) = \frac{p(\mathbf{X}|\mathbf{z}_n)p(\mathbf{z}_n)}{p(\mathbf{X})}.$$

• Using conditional independence property, we obtain:

$$p(\mathbf{z}_n|\mathbf{X}) = \frac{p(\mathbf{x}_1, .., \mathbf{x}_n|\mathbf{z}_n)p(\mathbf{x}_{n+1}, ..., \mathbf{x}_N|\mathbf{z}_n)p(\mathbf{z}_n)}{p(\mathbf{X})}$$

$$= \frac{p(\mathbf{x}_1, .., \mathbf{x}_n, \mathbf{z}_n)p(\mathbf{x}_{n+1}, ..., \mathbf{x}_N|\mathbf{z}_n)}{p(\mathbf{X})} = \frac{\alpha(\mathbf{z}_n)\beta(\mathbf{z}_n)}{p(\mathbf{X})}.$$



Inference of Hidden States

- Hence:

$$\gamma(\mathbf{z}_n) = \frac{p(\mathbf{x}_1, .., \mathbf{x}_n, \mathbf{z}_n)p(\mathbf{x}_{n+1}, ..., \mathbf{x}_N|\mathbf{z}_n)}{p(\mathbf{X})} = \frac{\alpha(\mathbf{z}_n)\beta(\mathbf{z}_n)}{p(\mathbf{X})}.$$

$$\alpha(\mathbf{z}_n) \equiv p(\mathbf{x}_1, .., \mathbf{x}_n, \mathbf{z}_n)$$

The joint probability of observing all of the data up to time n and z_n.

$$\beta(\mathbf{z}_n) \equiv p(\mathbf{x}_{n+1}, .., \mathbf{x}_N|\mathbf{z}_n).$$

The conditional probability of all future data from time n+1 to N.

- Each $\alpha(z_n)$ and $\beta(z_n)$ represent a set of K numbers, one for each of the possible settings of the 1-of-K binary vector z_n.

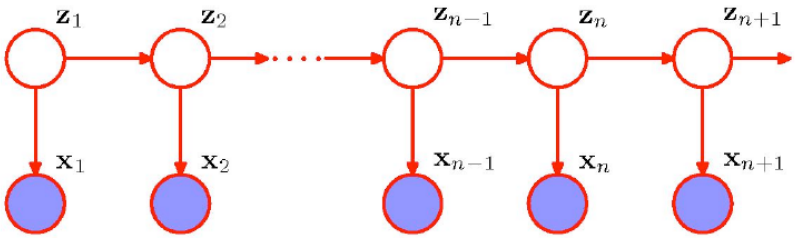
- We will derive efficient recursive algorithm, known as the alpha-beta recursion, or forward-backward algorithm.

The Forward (α) Recursion

- The forward recursion:

$$\begin{aligned}
\alpha(\mathbf{z}_n) &= p(\mathbf{x}_1, .., \mathbf{x}_n, \mathbf{z}_n) \\
&= p(\mathbf{x}_n|\mathbf{z}_n)p(\mathbf{x}_1, .., \mathbf{x}_{n-1}, \mathbf{z}_n) \\
&= p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} p(\mathbf{x}_1, .., \mathbf{x}_{n-1}, \mathbf{z}_{n-1}, \mathbf{z}_n) \\
&= p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} p(\mathbf{x}_1, .., \mathbf{x}_{n-1}, \mathbf{z}_{n-1})p(\mathbf{z}_n|\mathbf{z}_{n-1}) \\
&= p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} \alpha(\mathbf{z}_{n-1})p(\mathbf{z}_n|\mathbf{z}_{n-1})
\end{aligned}$$

Computational cost scales like $O(K^2)$.



- Observe:

$$p(\mathbf{X}) = \sum_{\mathbf{z}_N} \alpha(\mathbf{z}_N).$$

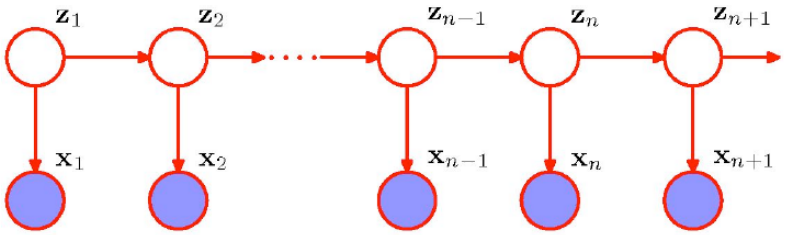
- This enables us to easily (cheaply) compute the desired likelihood.

The Forward (α) Recursion

- The forward recursion:

$$\begin{aligned}
\alpha(\mathbf{z}_n) &= p(\mathbf{x}_1, .., \mathbf{x}_n, \mathbf{z}_n) \\
&= p(\mathbf{x}_n|\mathbf{z}_n)p(\mathbf{x}_1, .., \mathbf{x}_{n-1}, \mathbf{z}_n) \\
&= p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} p(\mathbf{x}_1, .., \mathbf{x}_{n-1}, \mathbf{z}_{n-1}, \mathbf{z}_n) \\
&= p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} p(\mathbf{x}_1, .., \mathbf{x}_{n-1}, \mathbf{z}_{n-1})p(\mathbf{z}_n|\mathbf{z}_{n-1}) \\
&= p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} \alpha(\mathbf{z}_{n-1})p(\mathbf{z}_n|\mathbf{z}_{n-1})
\end{aligned}$$

Computational cost scales like $O(K^2)$.



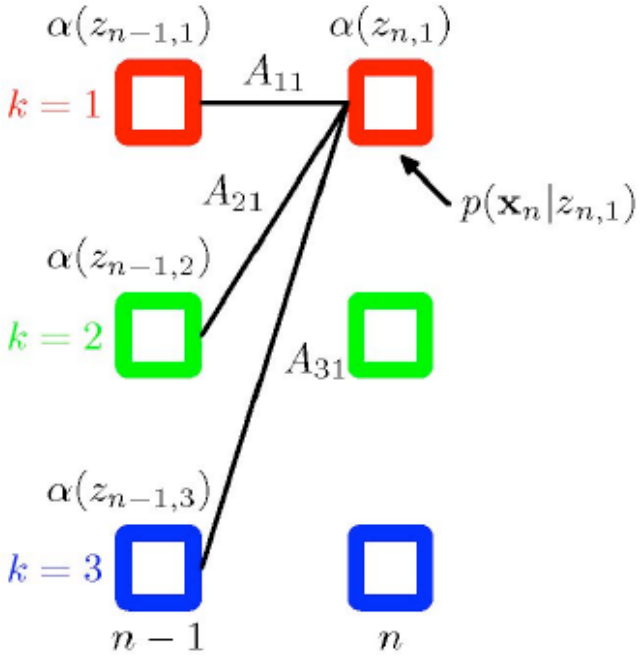
- Observe:

$$p(\mathbf{X}) = \sum_{\mathbf{z}_N} \alpha(\mathbf{z}_N).$$

- This enables us to easily (cheaply) compute the desired likelihood.

The Forward (α) Recursion

- Illustration of the forward recursion



Here $\alpha(z_{n,1})$ is obtained by
- Taking the elements $\alpha(z_{n-1},j)$
- Summing the up with weights A_{j1}, corresponding to $p(z_n \mid z_{n-1})$
- Multiplying by the data contribution $p(x_n \mid z_{n1})$.

$$\alpha(\mathbf{z}_n) = p(\mathbf{x}_n|\mathbf{z}_n) \sum_{\mathbf{z}_{n-1}} \alpha(\mathbf{z}_{n-1}) p(\mathbf{z}_n|\mathbf{z}_{n-1})$$

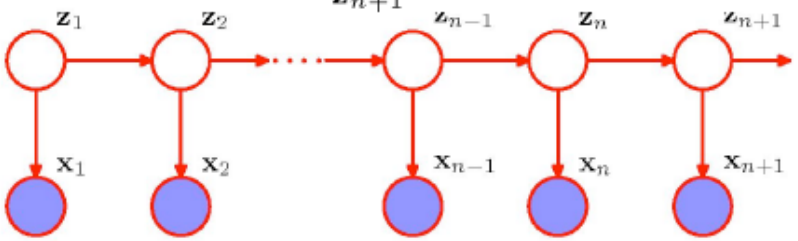
- The initial condition is given by:

$$\alpha(\mathbf{z}_1) = p(\mathbf{x}_1|\mathbf{z}_1)p(\mathbf{z}_1) = \prod_{k=1}^{K} \left[\pi_k p(\mathbf{x}_1|\boldsymbol{\phi}_k)\right]^{z_{1k}}.$$

The Backward (β) Recursion

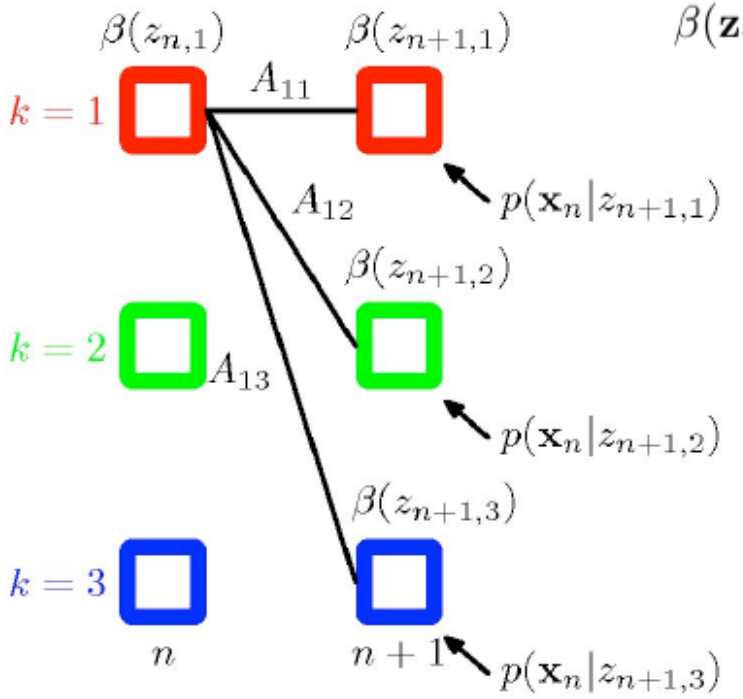
- There is also a simple recursion for $\beta(z_n)$:

$$
\begin{aligned}
\beta(\mathbf{z}_n) &= p(\mathbf{x}_{n+1}, .., \mathbf{x}_N | \mathbf{z}_n) \\
&= \sum_{\mathbf{z}_{n+1}} p(\mathbf{x}_{n+1}, ..., \mathbf{x}_N, \mathbf{z}_{n+1} | \mathbf{z}_n) \\
&= \sum_{\mathbf{z}_{n+1}} p(\mathbf{x}_{n+1}, ..., \mathbf{x}_N | \mathbf{z}_{n+1}, \mathbf{z}_n) p(\mathbf{z}_{n+1} | \mathbf{z}_n) \\
&= \sum_{\mathbf{z}_{n+1}} p(\mathbf{x}_{n+1}, ..., \mathbf{x}_N | \mathbf{z}_{n+1}) p(\mathbf{z}_{n+1} | \mathbf{z}_n) \\
&= \sum_{\mathbf{z}_{n+1}} p(\mathbf{x}_{n+2}, ..., \mathbf{x}_N | \mathbf{z}_{n+1}) p(\mathbf{x}_{n+1} | \mathbf{z}_{n+1}) p(\mathbf{z}_{n+1} | \mathbf{z}_n) \\
&= \sum_{\mathbf{z}_{n+1}} \beta(\mathbf{z}_{n+1}) p(\mathbf{x}_{n+1} | \mathbf{z}_{n+1}) p(\mathbf{z}_{n+1} | \mathbf{z}_n)
\end{aligned}
$$



The Backward (β) Recursion

- Illustration of the backward recursion



$$\beta(\mathbf{z}_n) = \sum_{\mathbf{z}_{n+1}} \beta(\mathbf{z}_{n+1}) p(\mathbf{x}_{n+1}|\mathbf{z}_{n+1}) p(\mathbf{z}_{n+1}|\mathbf{z}_n)$$

- Initial condition:

$$p(\mathbf{z}_N|\mathbf{X}) = \frac{\alpha(\mathbf{z}_N)\beta(\mathbf{z}_N)}{p(\mathbf{X})} = \frac{p(\mathbf{X},\mathbf{z}_N)\beta(\mathbf{z}_N)}{p(\mathbf{X})}.$$

- Hence:

$$\beta(\mathbf{z}_N) = 1.$$

Computing Likelihood

- Note that

$$\sum_{\mathbf{z}_n} \gamma(\mathbf{z}_n) = \sum_{\mathbf{z}_n} p(\mathbf{z}_n|\mathbf{X}) = 1.$$

- We can compute the likelihood at any time using α - β recursion:

$$p(\mathbf{X}|\theta) = \sum_{\mathbf{z}_n} \alpha(\mathbf{z}_n)\beta(\mathbf{z}_n).$$

- In the forward calculation we proposed originally, we did this at the final time step n = N.

$$p(\mathbf{X}|\theta) = \sum_{\mathbf{z}_N} \alpha(\mathbf{z}_N).$$

Because $\beta(z_n)=1$.

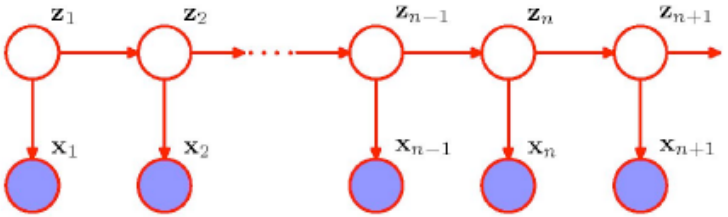
- This is a good way to check your code!

Two-Frame Inference

• We will also need the cross-time statistics for adjacent time steps:

$$\begin{aligned}\xi(\mathbf{z}_{n-1},\mathbf{z}_n) &= p(\mathbf{z}_{n-1},\mathbf{z}_n|\mathbf{X})\\ &= \frac{p(\mathbf{X}|\mathbf{z}_{n-1},\mathbf{z}_n)p(\mathbf{z}_{n-1},\mathbf{z}_n)}{p(\mathbf{X})}\\ &= \frac{p(\mathbf{x}_1,..,\mathbf{x}_{n-1}|\mathbf{z}_{n-1})p(\mathbf{x}_n|\mathbf{z}_n)p(\mathbf{x}_{n+1},...,\mathbf{x}_N|\mathbf{z}_n)p(\mathbf{z}_n|\mathbf{z}_{n-1})p(\mathbf{z}_{n-1})}{p(\mathbf{X})}\\ &= \frac{\alpha(\mathbf{z}_{n-1})p(\mathbf{x}_n|\mathbf{z}_n)p(\mathbf{z}_n|\mathbf{z}_{n-1})\beta(\mathbf{z}_n)}{p(\mathbf{X})}.\end{aligned}$$

• This is a K × K matrix with elements ξ(i,j) representing the expected number of transitions from state i to state j that begin at time n-1, given all the observations.



• It can be computed with the same α and β recursions.

EM algorithm

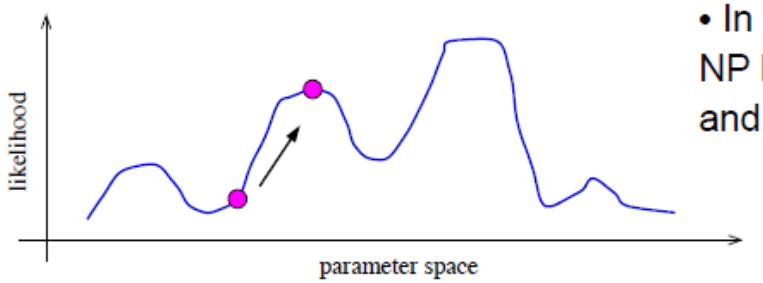
- Intuition: if only we knew the true state path then ML parameter estimation would be trivial.
- E-step: Compute the posterior distribution over the state path using α - β recursion (dynamic programming):

$$p(\mathbf{Z}|\mathbf{X}, \theta^{old}).$$

- M-step: Maximize the expected complete data log-likelihood (parameter re-estimation):

$$Q(\theta, \theta^{old}) = \sum_{\mathbf{Z}} p(\mathbf{Z}|\mathbf{X}, \theta^{old}) \log p(\mathbf{X}, \mathbf{Z}|\theta).$$

- We then iterate. This is also known as a Baum-Welch algorithm (special case of EM).



- In general, finding the ML parameters is NP hard, so initial conditions matter a lot and convergence is hard to tell.

Complete Data Log-likelihood

- Complete data log-likelihood takes form:

$$\log p(\mathbf{X}, \mathbf{Z}|\theta) = \log \left[p(\mathbf{z}_1|\boldsymbol{\pi}) \prod_{n=2}^{N} p(\mathbf{z}_n|\mathbf{z}_{n-1}, A) \prod_{n=1}^{N} p(\mathbf{x}_n|\mathbf{z}_n, \boldsymbol{\phi}) \right]$$

$$= \log \left[\prod_{k=1}^{K} \pi_k^{z_{1k}} \prod_{n=2}^{N} \prod_{k=1}^{K} \prod_{j=1}^{K} A_{jk}^{z_{n-1,j} z_{nk}} \prod_{n=1}^{N} \prod_{k=1}^{K} p(\mathbf{x}_n|\mathbf{z}_n)^{z_{nk}} \right]$$

$$= \sum_{k=1}^{K} z_{1k} \log \pi_k + \sum_{n=2}^{N} \sum_{k=1}^{K} \sum_{j=1}^{K} [z_{n-1,j} z_{nk}] \log A_{jk} + \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \log p(\mathbf{x}_n|\mathbf{z}_n).$$

transition model

observation model

- Statistics we need from the E-step are:

$$\gamma(\mathbf{z}_n) = p(\mathbf{z}_n|\mathbf{X}).$$

$$\xi(\mathbf{z}_{n-1}, \mathbf{z}_n) = p(\mathbf{z}_{n-1}, \mathbf{z}_n|\mathbf{X}).$$

Expected Complete Data Log-likelihood

- The complete data log-likelihood takes form:

$$Q(\theta, \theta^{old}) = \sum_{\mathbf{Z}} p(\mathbf{Z}|\mathbf{X}, \theta^{old}) \log p(\mathbf{X}, \mathbf{Z}|\theta).$$

$$= \sum_{k=1}^{K} \gamma(z_{1k}) \log \pi_k + \sum_{n=2}^{N} \sum_{k=1}^{K} \sum_{j=1}^{K} \xi(z_{n-1,j} z_{nk}) \log A_{jk} + \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma_{nk} \log p(\mathbf{x}_n|\mathbf{z}_n).$$

- Hence in the E-step we evaluate:

$$\gamma(\mathbf{z}_n) = p(\mathbf{z}_n|\mathbf{X}).$$

$$\xi(\mathbf{z}_{n-1}, \mathbf{z}_n) = p(\mathbf{z}_{n-1}, \mathbf{z}_n|\mathbf{X}).$$

- In the M-step we optimize Q with respect to parameters: $\theta = \{\boldsymbol{\pi}, A, \boldsymbol{\phi}\}$.

Parameter Estimation

- Initial state distribution: expected number of times in state k at time 1:

$$\pi_k^{new} = \frac{\gamma(z_{1k})}{\sum_{j=1}^{K} \gamma(z_{1j})}.$$

- Expected # of transitions from state j to k which begin at time n-1:

$$\xi(\mathbf{z}_{n-1,j}, \mathbf{z}_{n,k}) = p(\mathbf{z}_{n-1,j}, \mathbf{z}_{n,k}|\mathbf{X}),$$

so the estimated transition probabilities are:

$$A_{jk}^{new} = \frac{\sum_{n=2}^{N} \xi(z_{n-1,j}, z_{nk})}{\sum_{l=1}^{K} \sum_{n=2}^{N} \xi(z_{n-1,j}, z_{nl})}.$$

- The EM algorithm must be initialized by choosing starting values for π and **A**.

- Note that any elements of π or **A** that initially are set to zero will remain zero in subsequent EM updates.

Parameter Estimation: Emission Model

- For the case of discrete multinomial observed variables, the observation model takes form:

$$p(\mathbf{x}_n|\mathbf{z}_n,\boldsymbol{\phi}) = \prod_{i=1}^{D}\prod_{k=1}^{K}\mu_{ik}^{x_{ni}z_{nk}}.$$

Same as fitting Bernoulli mixture model.

- And the corresponding M-step update: $\mu_{ik}^{new} = \dfrac{\sum_{n=1}^{N}\gamma(z_{nk})x_{ni}}{\sum_{n=1}^{N}\gamma(z_{nk})}.$

- For the case of the Gaussian emission model:

$$p(\mathbf{x}_n|\mathbf{z}_n,\boldsymbol{\phi}) = \prod_{k=1}^{K}\mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)^{z_{nk}}.$$

Remember:

$$\gamma(\mathbf{z}_n) = p(\mathbf{z}_n|\mathbf{X}).$$

- And the corresponding M-step updates:

$$\boldsymbol{\mu}_k^{new} = \frac{1}{N_k}\sum_n \gamma(z_{nk})\mathbf{x}_n, \quad N_k = \sum_n \gamma(z_{nk}),$$

Same as fitting a Gaussian mixture model.

$$\boldsymbol{\Sigma}_k^{new} = \frac{1}{N_k}\sum_{n=1}^{N}\gamma(y_{nk})(\mathbf{x}_n-\boldsymbol{\mu}_k)(\mathbf{x}_n-\boldsymbol{\mu}_k)^T,$$

Viterbi Decoding

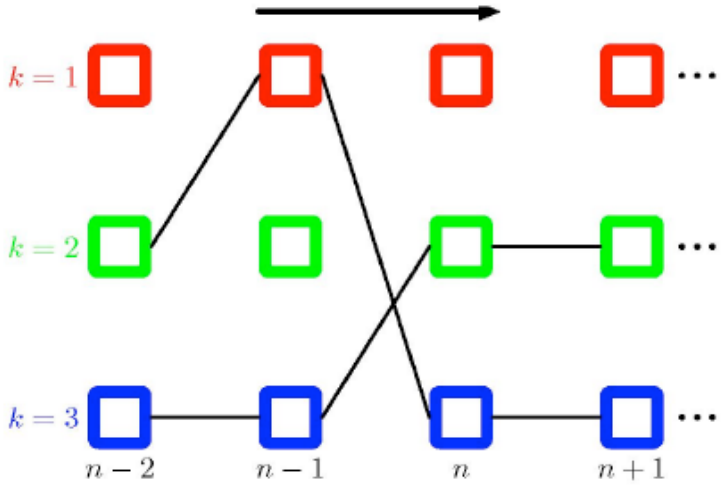
- The numbers $\gamma(z_n)$ above gave the probability distribution over all states at any time.

$$\gamma(\mathbf{z}_n) = p(\mathbf{z}_n|\mathbf{X}).$$

- By choosing the state $\gamma^*(z_n)$ with the largest probability at each time, we can make an "average" state path. This is the path with the maximum expected number of correct states.
- To find the single best path, we do Viterbi decoding which is Bellman's dynamic programming algorithm applied to this problem.
- The recursions look the same, except with max instead of $\sum$.
- Same dynamic programming trick: instead of summing, we keep the term with the highest value at each node.
- There is also a modified EM (Baum-Welch) training based on the Viterbi decoding. Like K-means instead of mixtures of Gaussians.
- Relates to the max-sum algorithm for tree structured graphical models.

Viterbi Decoding

• A fragment of the HMM lattice showing two possible paths:



• Viterbi decoding efficiently determines the most probable path from the exponentially many possibilities.

• The probability of each path is given by the product of the elements of the transition matrix A_{jk}, along with the emission probabilities associated with each node in the path.

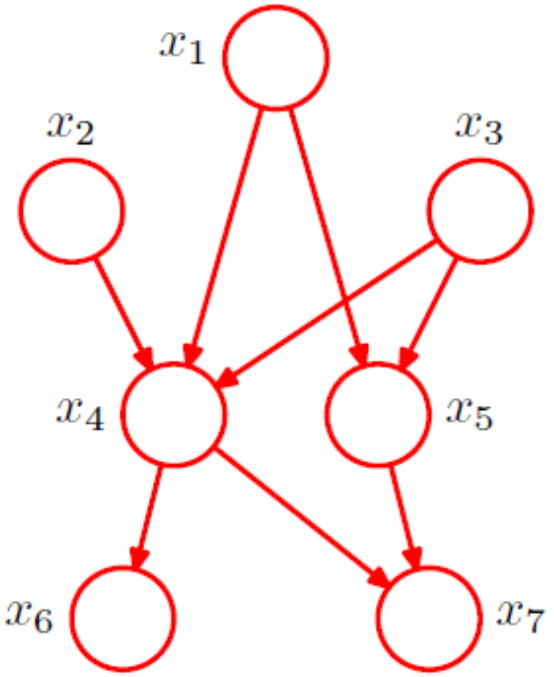
Applications of HMMs

- Speech recognition.
- Language modeling
- Motion video analysis/tracking.
- Protein sequence and genetic sequence alignment and analysis.
- Financial time series prediction.

Other Graphical Models

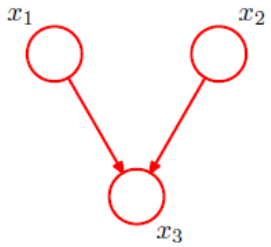
- Bayesian Network

$$p(x_1)p(x_2)p(x_3)p(x_4|x_1,x_2,x_3)p(x_5|x_1,x_3)p(x_6|x_4)p(x_7|x_4,x_5)$$

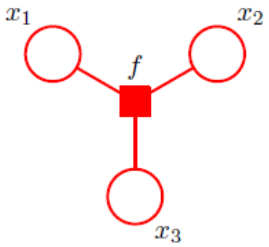


Conditional Random Field

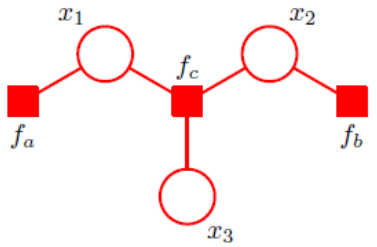
- Factor Graph



$p(x) = p(x_1)p(x_2)$
$p(x_3|x_1, x_2)$

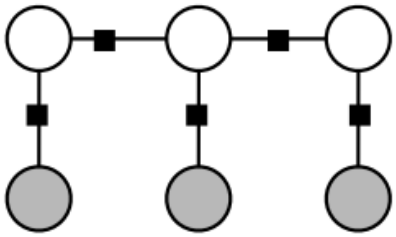


$f(x_1, x_2, x_3) =$
$p(x_1)p(x_2)p(x_3|x_1, x_2)$

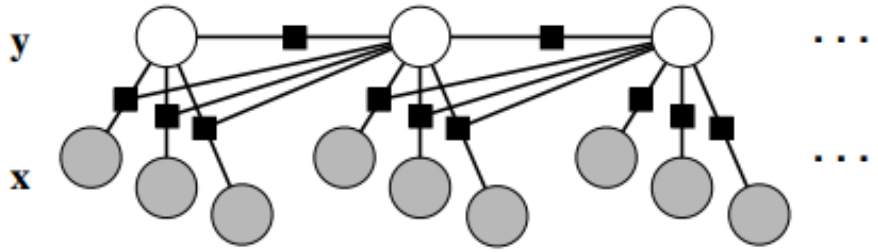


$f_a(x_1) = p(x_1)$
$f_b(x_2) = p(x_2)$
$f_c(x_1, x_2, x_3) = p(x_3|x_2, x_1)$

- Conditional Random Field



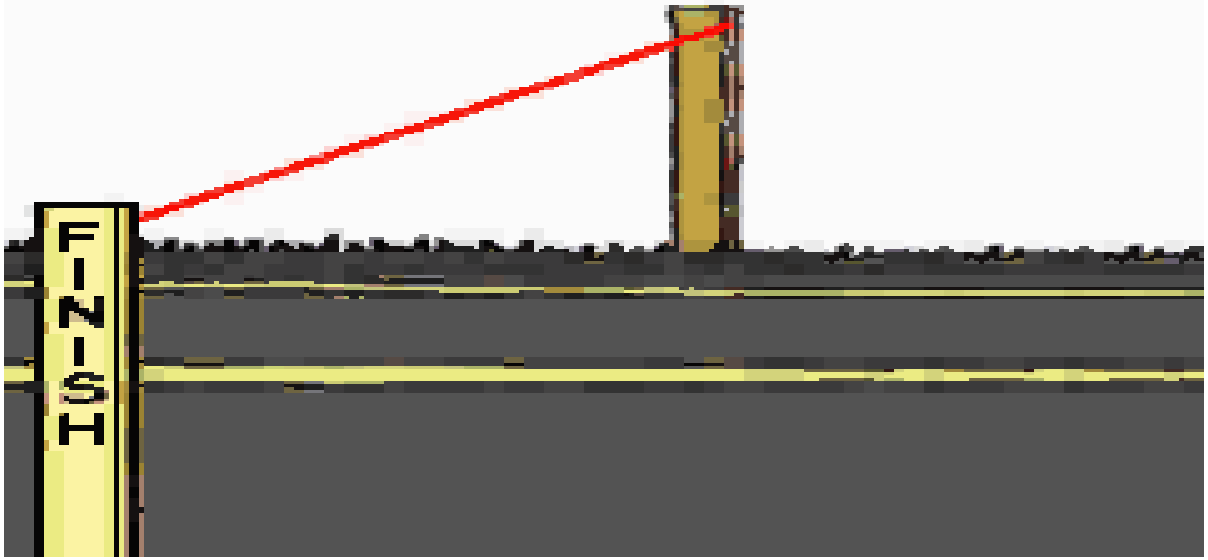
Linear-chain CRFs



More Bayes Inference Methods

- Belief Propagation
- Variational Inference
- Mean Field
- Monte Carlo Sampling
- …

Kevin P. Murphy, Probabilistic Machine Learning: Advanced Topics, MIT Press, 2023



END